



How to Set Up Kubernetes Network Policies



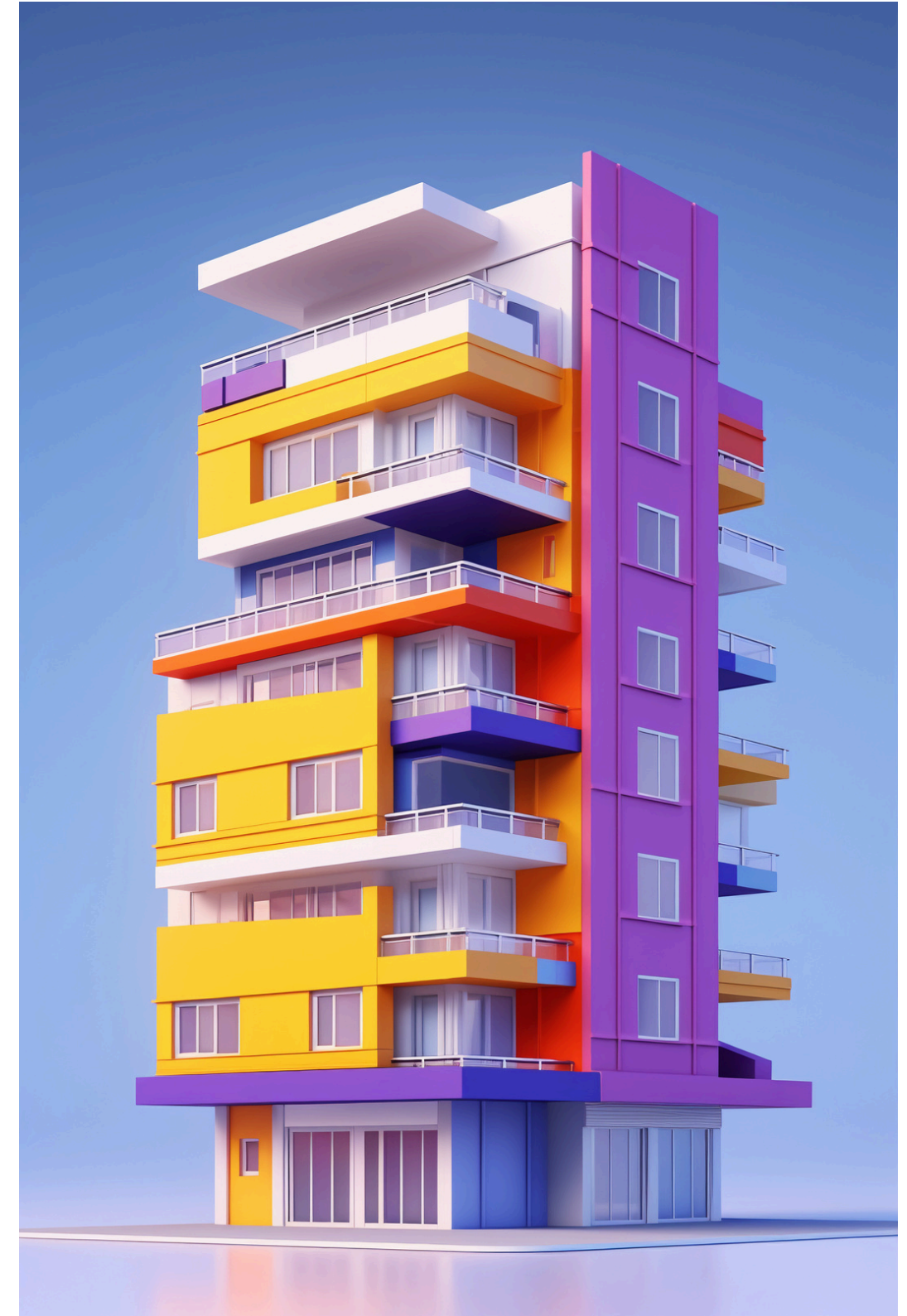
ramesh
(app-pod)



suresh
(db-pod)



Cluster





doors are fully opened

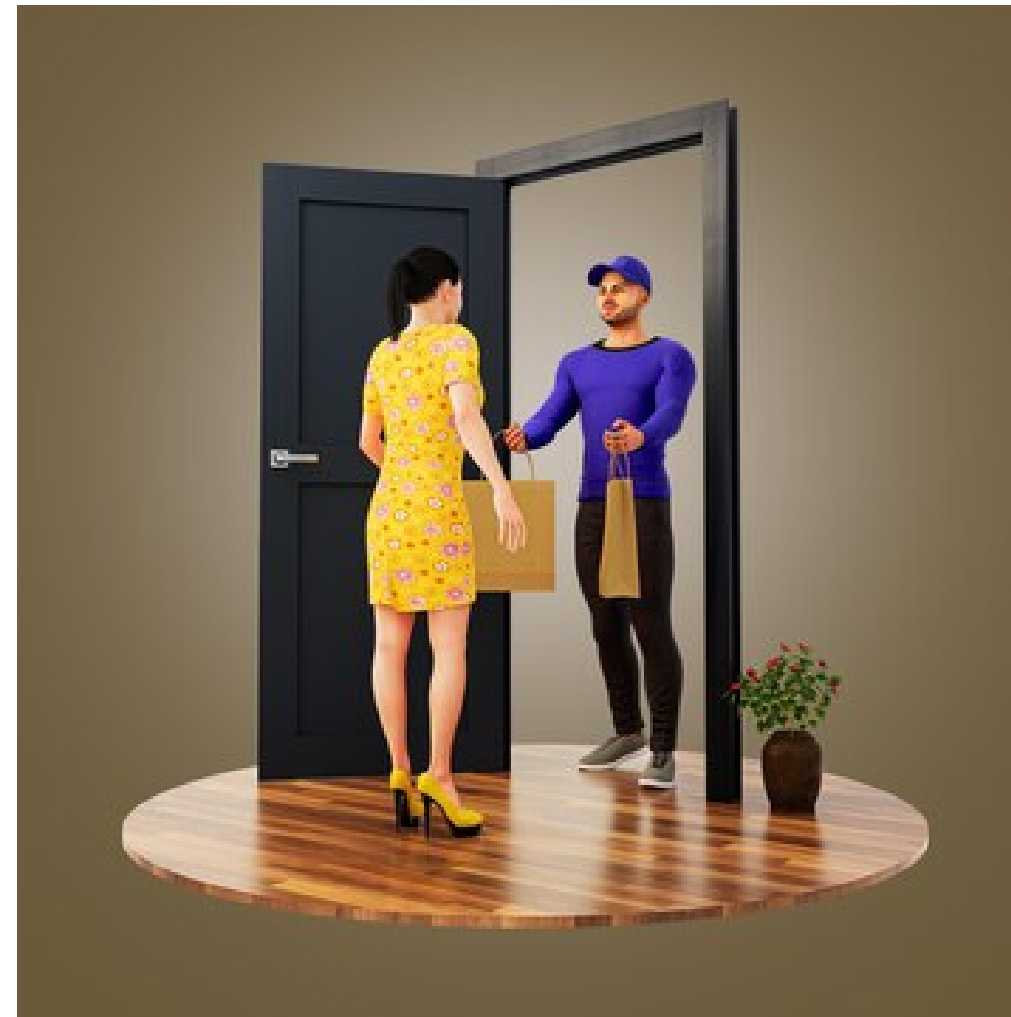


so both people can easily meet

zomato
(log-pods)



zepto
(argocd-pods)



PDB
(monitor-pods)



delivery boys also entering into room without permission

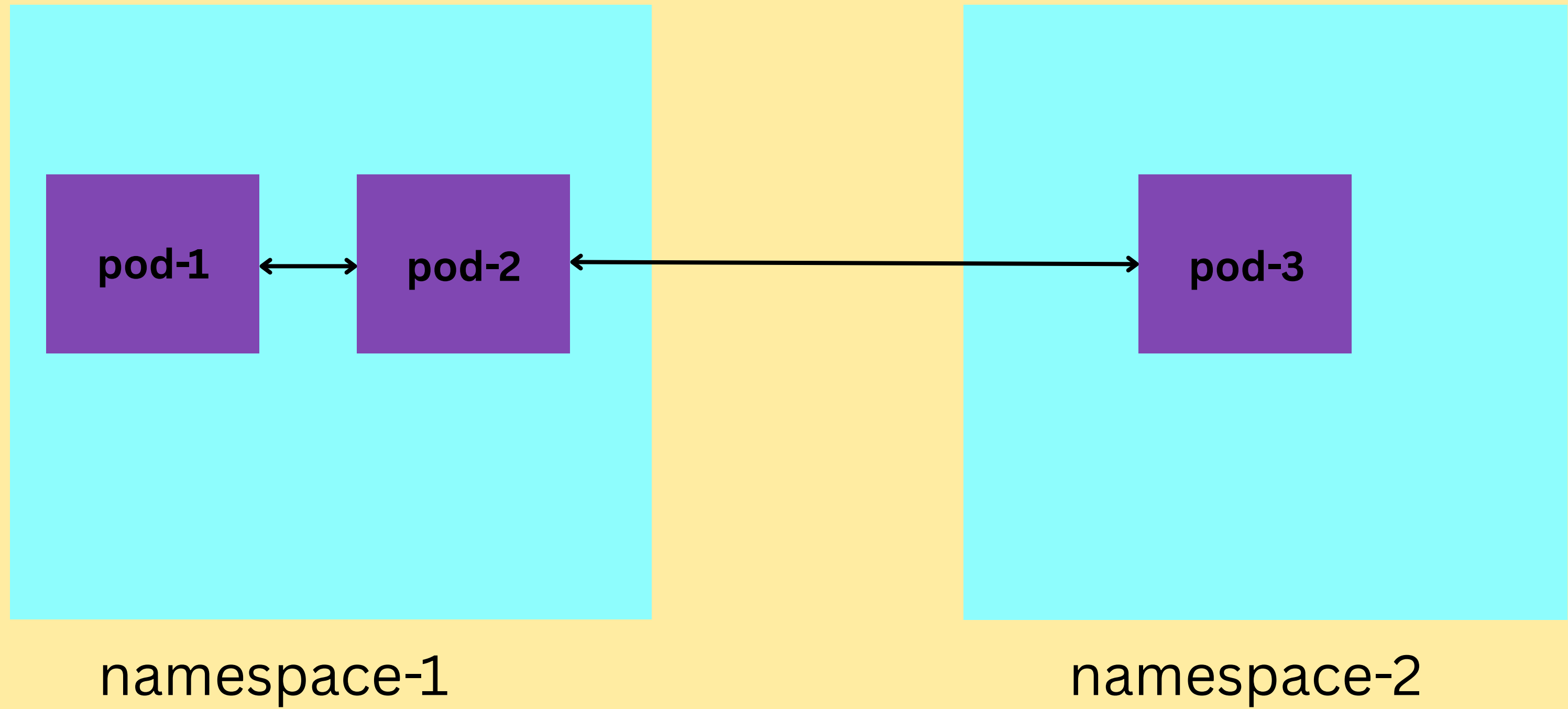


Lets install some advanced security doors

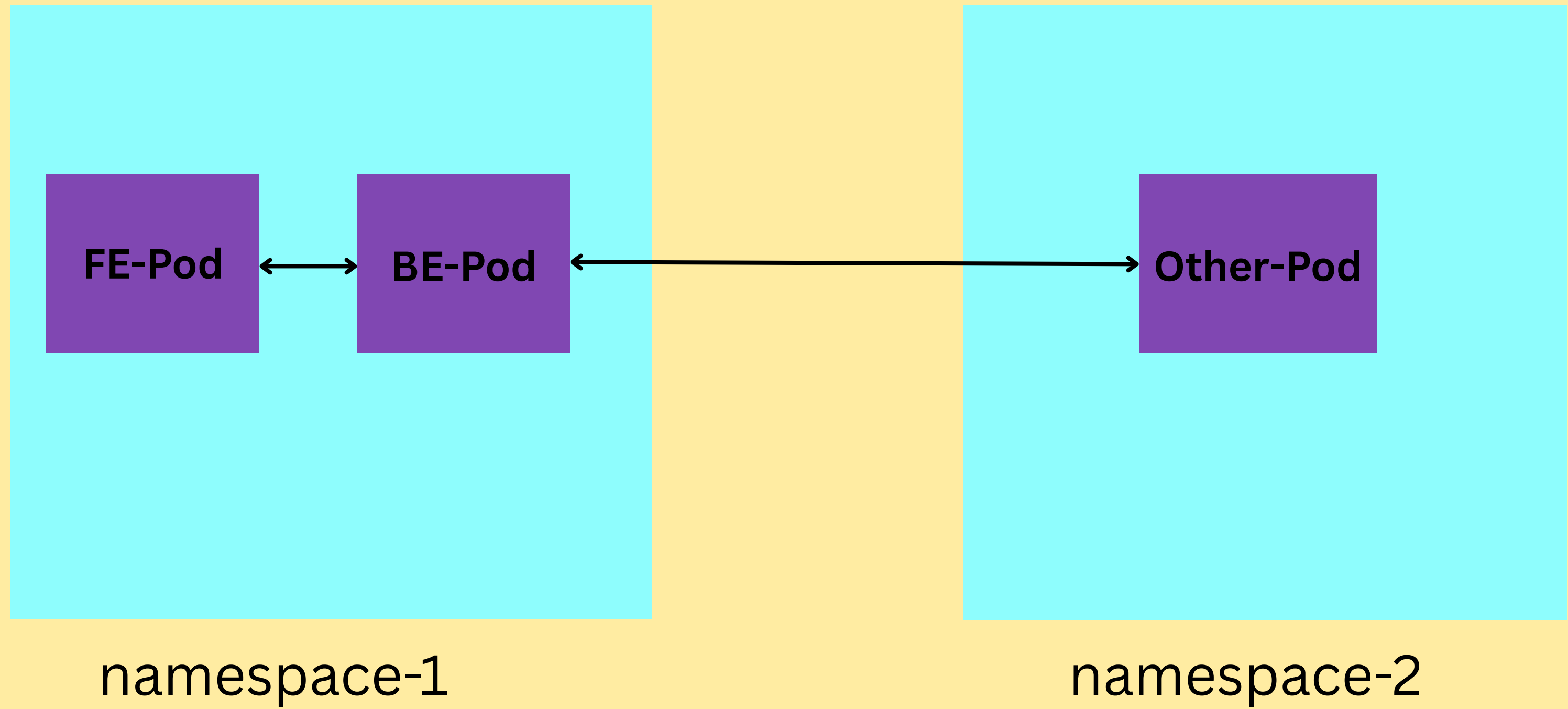


network policies

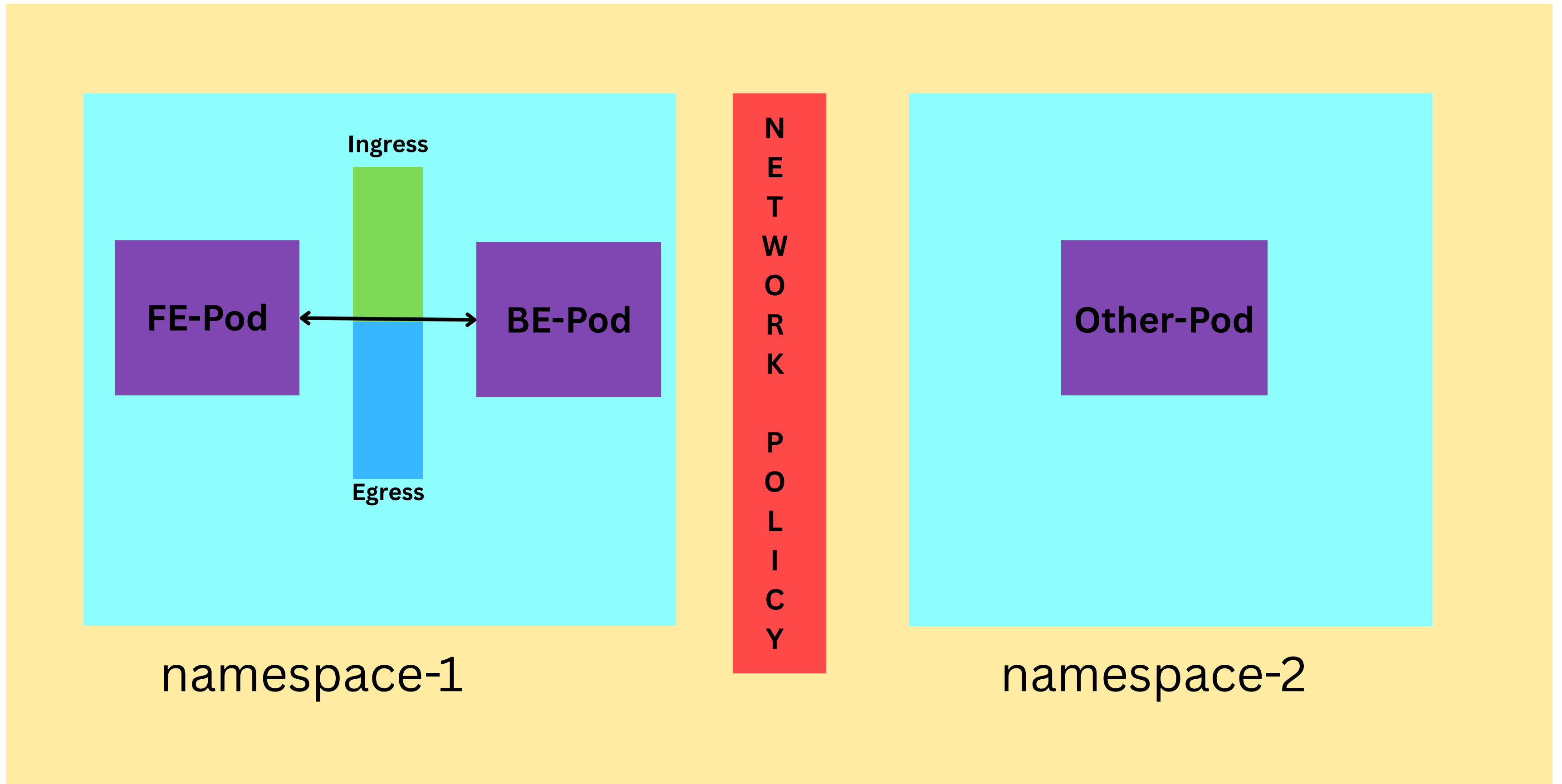
Cluster



Cluster



Cluster



What is a Network Policy?

Network Policies in Kubernetes are like firewall rules for pods. They control which pods can communicate with each other and with other network endpoints (e.g., external services).

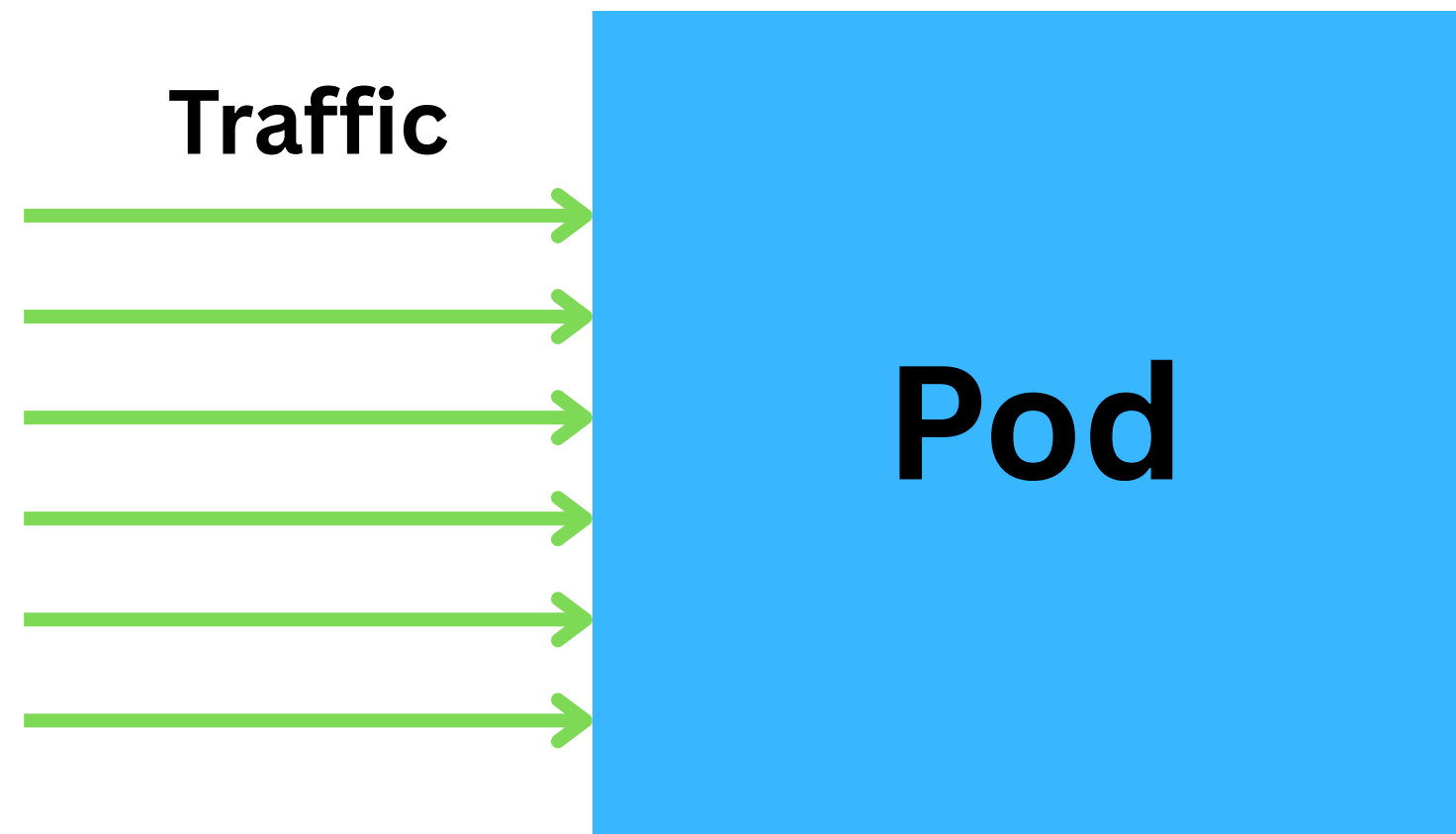
These policies are implemented **at the network level** by Kubernetes-compatible **Container Network Interface (CNI) plugins**, such as:

- Calico
- Cilium
- Weave Net

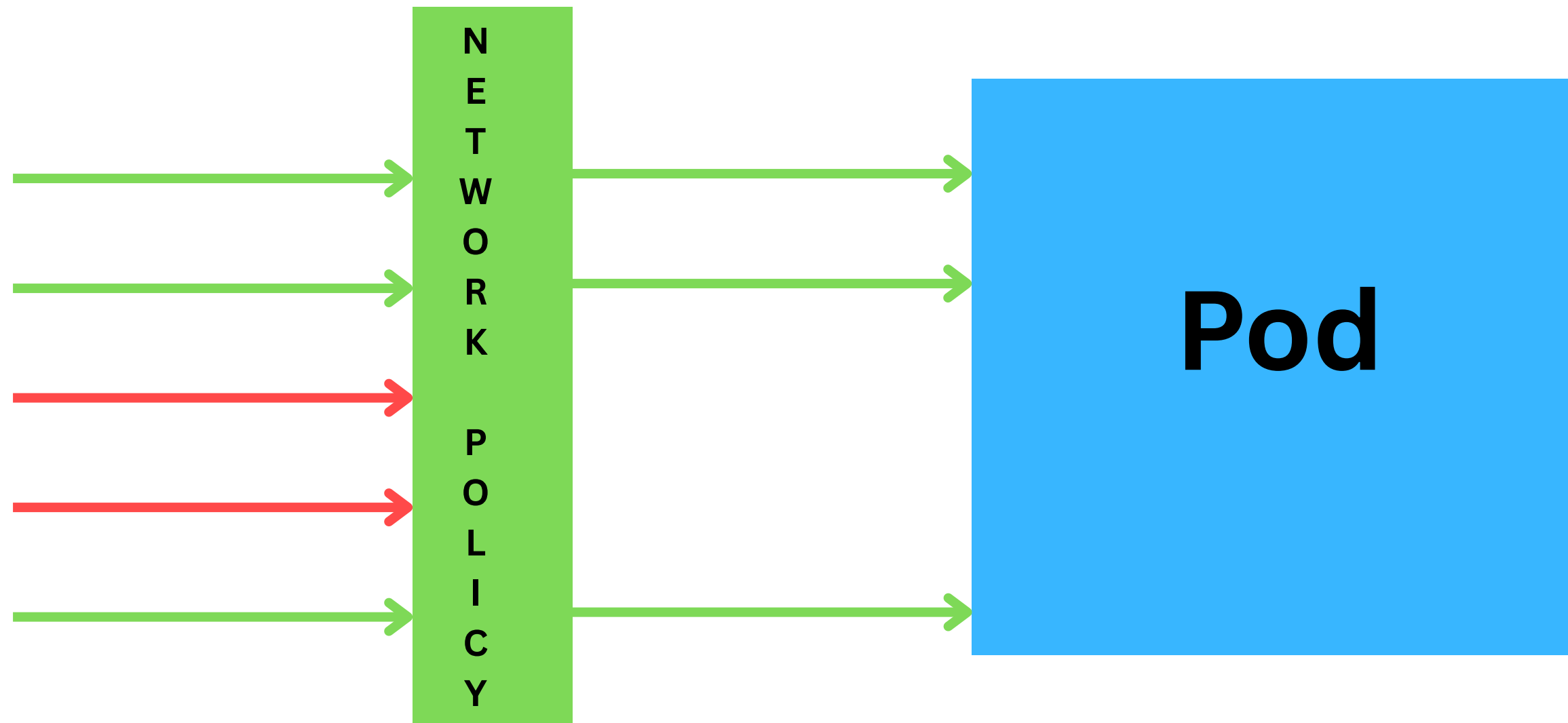


Order of Evaluation

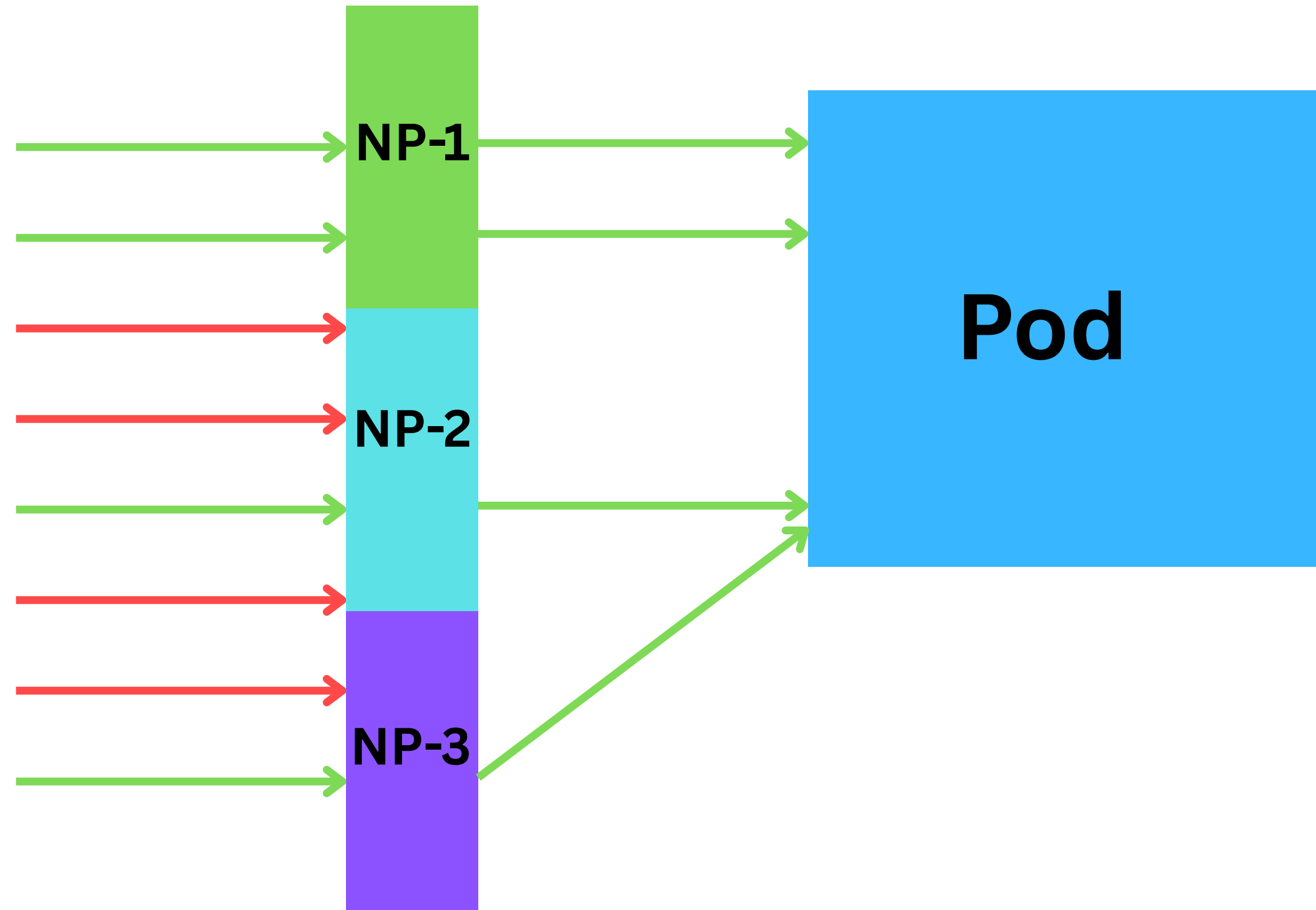
1. If a pod has no policy, all traffic is allowed



2. If a pod has policy, then required traffic only allowed



3. If a pod has one or more policies, only allowed traffic can pass



Some Technical Terms to Remember:

INGRESS RULE: Who can come into the pod



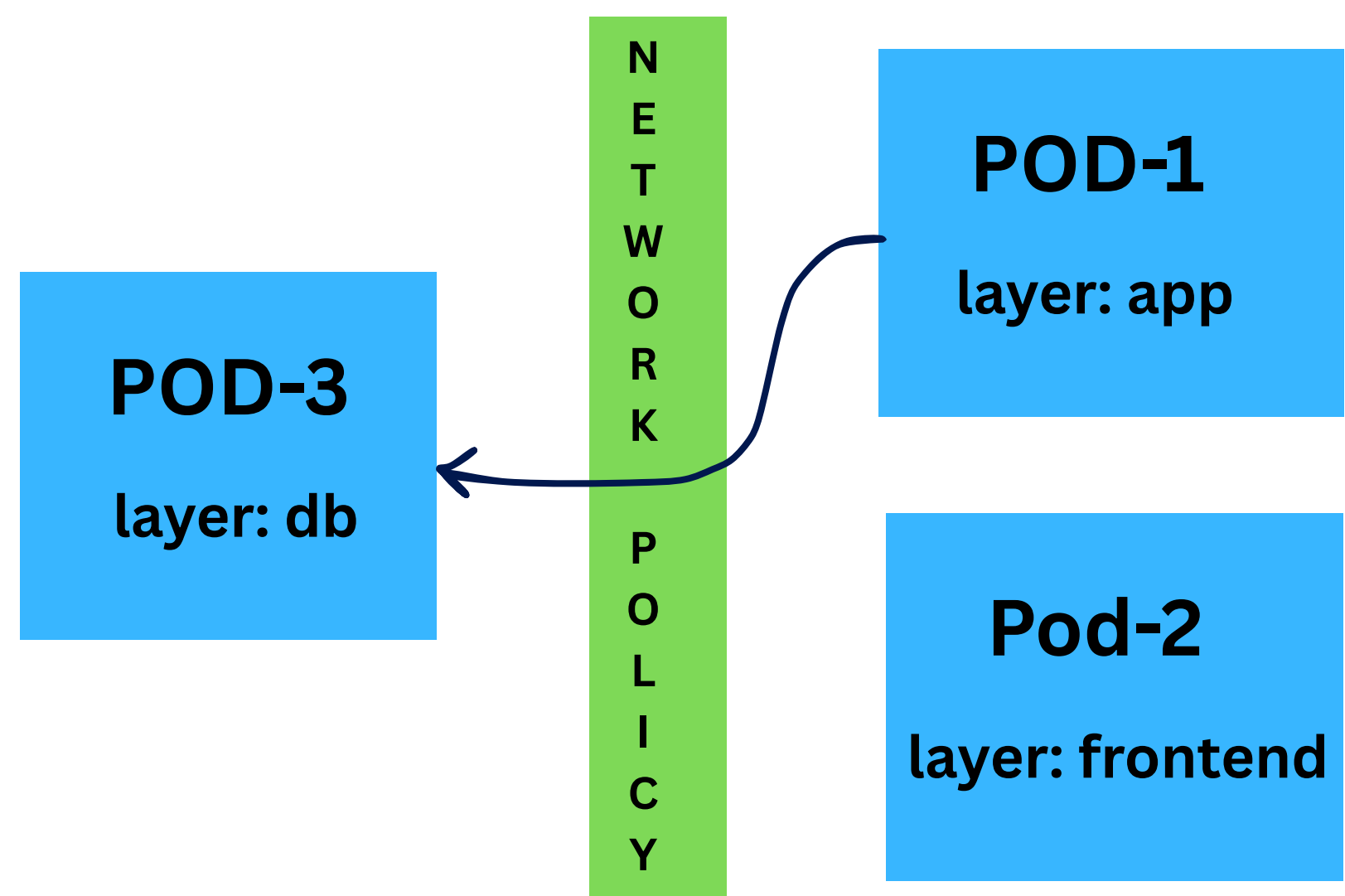
EGRESS RULE: Where the pod can go out



Some Technical Terms to Remember:

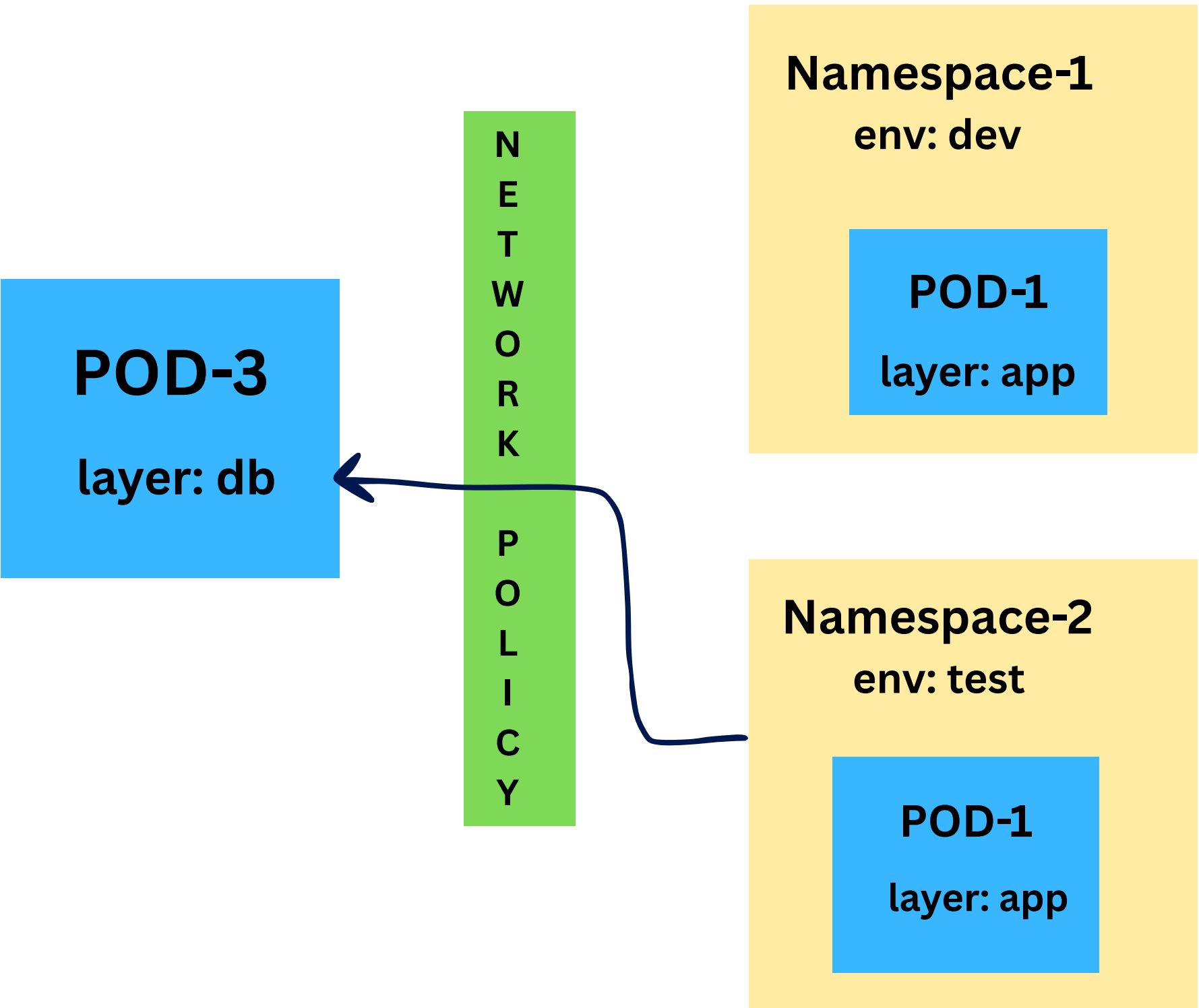
POD SELECTOR:

Target pods to which the policy applies



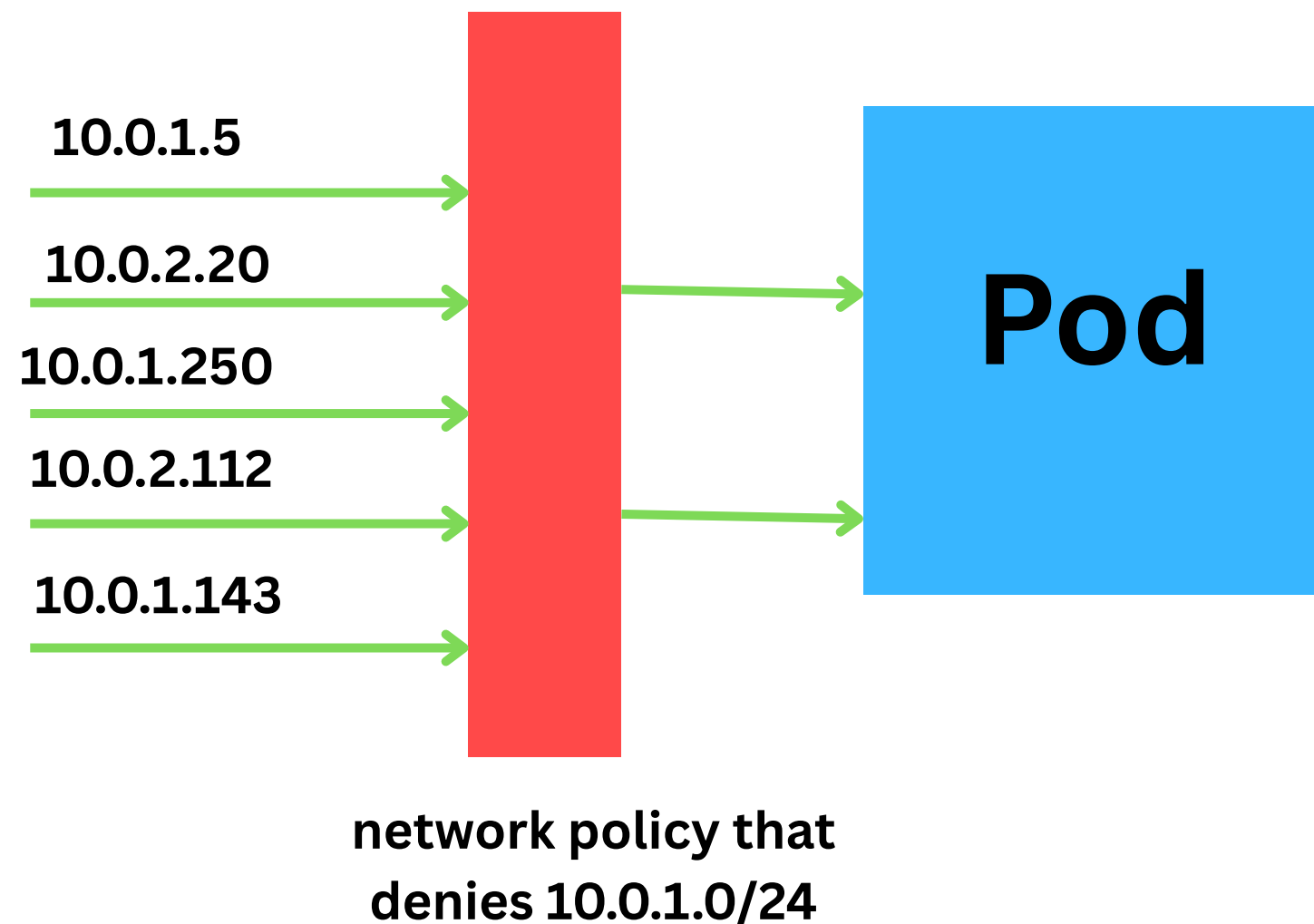
Ex: The pods which has label called “layer: app” they only can contact with db pod

NAMESPACE SELECTOR: Limit by namespace

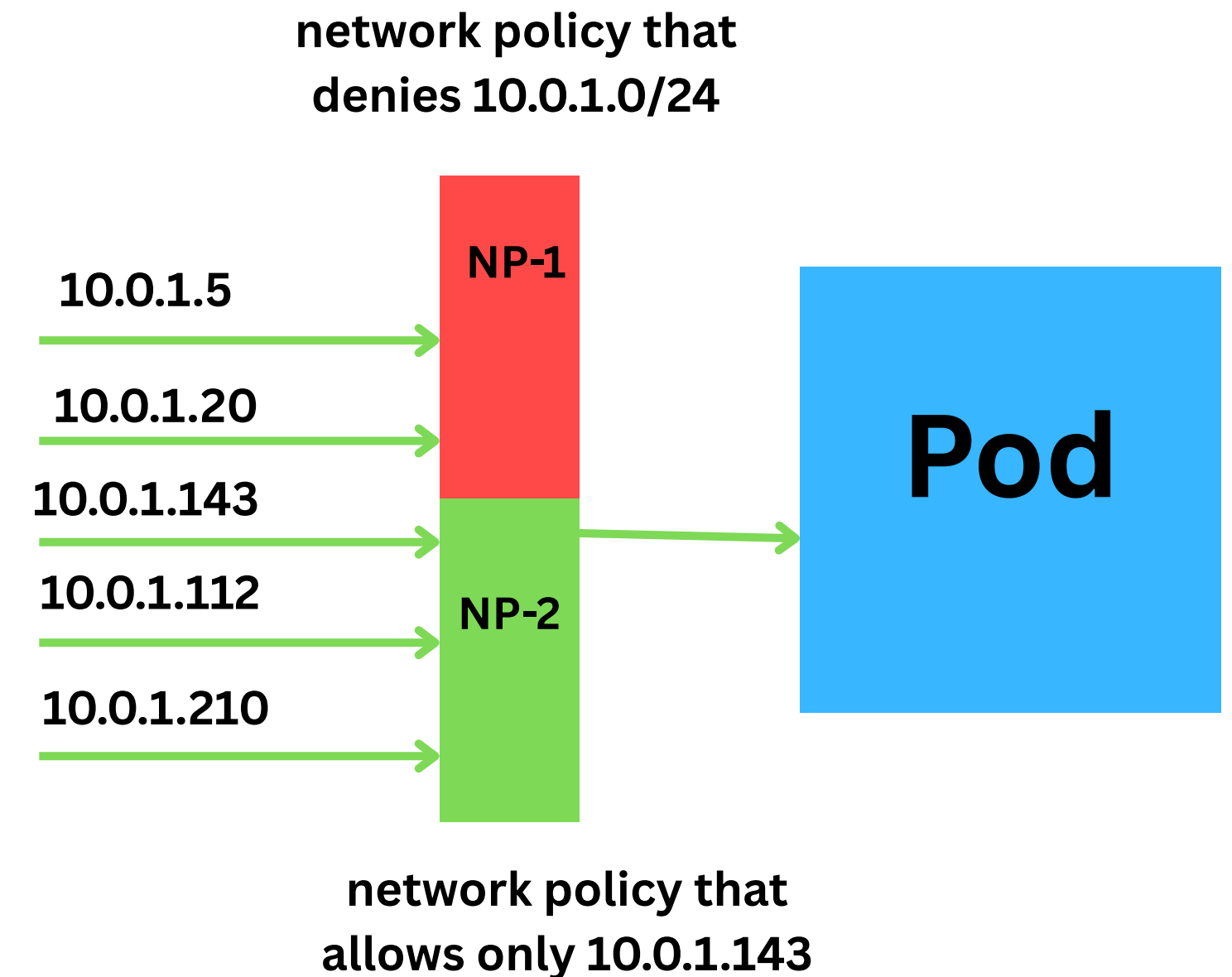


Some Technical Terms to Remember:

IP BLOCK: Allow/deny IP ranges



Ex-1: It denies all IP's with in the subnet 10.0.1.0/24



Ex-1: Here, NP-1 Denies all IP's from 10.0.1.0/24 but NP-2 Allows only 1 IP (10.0.1.143)

Denies all IP's

```
---  
apiVersion: networking.k8s.io/v1  
kind: NetworkPolicy  
metadata:  
  name: deny-all  
spec:  
  podSelector: {}  
  policyTypes:  
    - Ingress  
    - Egress
```


Allow Pods based on Labels:

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-nginx
spec:
  podSelector:
    matchLabels:
      app: nginx
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              project: test
          podSelector:
            matchLabels:
              app: swiggy

  ports:
    - port: 80
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: client-egress
spec:
  podSelector:
    matchLabels:
      app: swiggy
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              project: test
          podSelector:
            matchLabels:
              app: nginx

  ports:
    - port: 80
```


Common Interview Questions:

- **What is a Kubernetes NetworkPolicy?**
 - A NetworkPolicy is a Kubernetes resource that controls the traffic flow at the IP address or port level between pods and/or namespaces. It uses labels to define rules that allow or deny ingress and egress traffic.
- **What are the types of traffic a NetworkPolicy can control?**
 - Ingress: Traffic coming into a pod.
 - Egress: Traffic going out from a pod.
- **Does Kubernetes apply any default network policies?**
 - No, by default Kubernetes allows all traffic between pods. Only when a NetworkPolicy is applied does Kubernetes start restricting traffic.
- **What is the default behavior when multiple NetworkPolicies apply to a pod?**
 - They are additive. If multiple policies apply to a pod, the union of the rules is applied traffic is allowed if any policy allows it.

Scenario-Based Questions:

- **A pod suddenly loses connectivity to other services. What steps would you take to troubleshoot if a NetworkPolicy is the cause?**
 - Check if any NetworkPolicy applies to the pod (*kubectl get netpol*).
 - Examine the pod's labels and match with podSelector in policies.
 - Look for deny-all policies with no ingress or egress rules.
 - Verify whether the CNI plugin supports policies.
 - Use *kubectl describe netpol* and *kubectl describe pod pod-name* to inspect.
 - Test connectivity using temporary pods (*kubectl run tmp --rm -it --image=busybox sh*).

Scenario-Based Questions:

- **You added a NetworkPolicy, and now some services are unreachable. What could be wrong?**
 - The policy might be too restrictive (e.g., no matching from rules).
 - namespaceSelector or podSelector might not be matching anything.
 - Port mismatch in the policy.
 - The CNI plugin might not support NetworkPolicies.
 - No egress rule for DNS or external traffic.

Troubleshooting-Based Questions:

- **What are some best practices when designing NetworkPolicies?**
 - Start with default-deny for ingress and egress.
 - Allow only necessary communication.
 - Group pods using labels wisely.
 - Keep policies modular (per service/pod group).
 - Use namespaces to isolate workloads.
 - Use tools like Calico or Cilium for observability and debugging.

Troubleshooting-Based Questions:

- **How would you validate if your NetworkPolicies are working as expected?**
 - Use network policy simulators (e.g., Cilium Hubble, Calico Policy Audit).
 - Deploy test pods to simulate traffic (e.g., busybox, netshoot).
 - Use tools like curl, telnet, or nc to test connectivity.
 - Review CNI plugin logs and policy matches.

